

双路高速 AD 实验

ADC (Analog to Digital Converter 即模数转换器) 是大多数系统中必不可少的组成部件, 用于将连续的模拟信号转换成离散的数字信号, 它们是连接模拟电路和数字电路必不可少的桥梁。在很多场合下, ADC 的转换速度甚至直接决定了整个系统的运行速度。本章我们将使用双路高速 AD 模块来将外部的模拟信号转换成数字信号, 并在 ILA 中查看数字信号的波形。

本章包括以下几个部分:

- 1.1 简介
- 1.2 硬件设计
- 1.3 实验任务
- 1.4 程序设计
- 1.5 下载验证
- 1.6 本章总结
- 1.7 拓展训练

1.1 简介

本章我们使用的双路 AD 模块是正点原子推出的一款高速双路模数转换模块 (ATK_DUAL_HS_AD), 高速 AD 转换芯片由思瑞浦微电子科技(苏州)股份有限公司(简称 3PEAK)生产的国产芯片, 型号是 3PA1030。ATK_DUAL_HS_AD 模块的硬件结构图如下图所示。

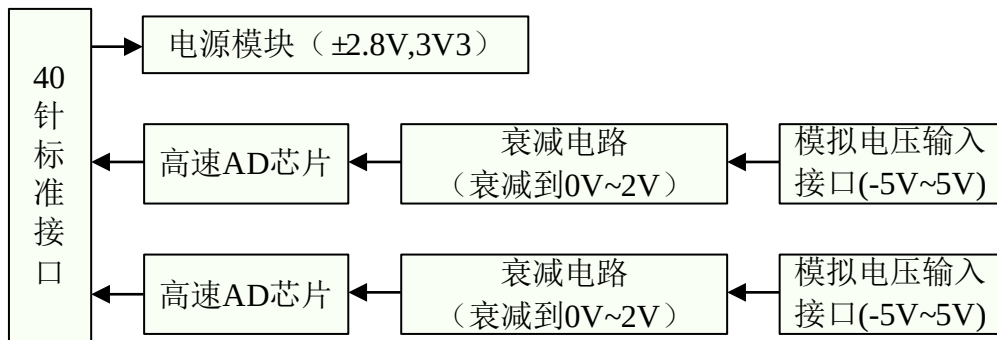


图 1.1.1 ATK_DUAL_HS_AD 模块硬件结构图

ATK_DUAL_HS_AD 模块使用的 AD 芯片型号为 3PA1030, 其输入模拟电压转换范围是 0V~2V, 所以电压输入端需要先经过电压衰减电路, 使输入的 -5V~+5V 之间的电压衰减到 0V~2V 之间, 然后再经过 3PA1030 芯片将模拟电压信号转换成数字信号。

下面我们介绍下这款芯片。

3PA1030 是一款思瑞浦推出的单电压芯片, 其模数转换位数为 10 位 (即分辨率为 $2^{10}=1024$), 最大转换速度为 50MSPS (每秒采样百万次, Million Samples per Second), 集成片上采样保持放大器和基准电压源。具有高性能低功耗的特点。

3PA1030 的内部功能框图如下图所示:

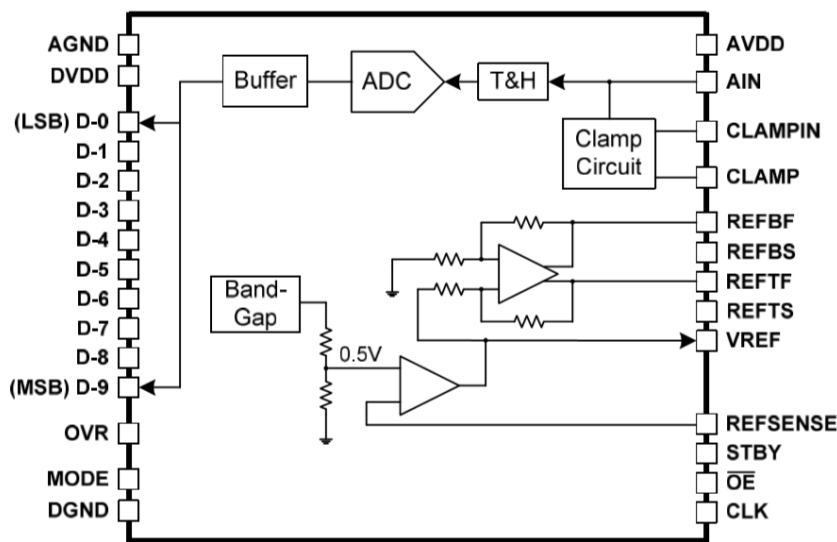


Figure 1. Functional Block Diagram

图 1.1.2 3PA1030 的内部功能框图

3PA1030 在时钟 (CLK) 的驱动下工作, 3PA1030 内置片内采样保持放大器 (SHA), 同时采用多级差分流水线架构, 保证了 50MSPS 的数据转换速率下全温度范围内无失码; 3PA1030 内部集成了基准源, 根据系统需要也可以选择外部高精度基准满足系统的要求。

3PA1030 输出的数据以二进制格式表示, 当输入的模拟电压超出量程时, 会拉高 OVR 信号; 当输入的模拟电压在量程范围内时, OVR 信号为低电平, 因此可以通过 OVR 信号来判断输入的模拟电压是否在测

量范围内。另外 3PA1030 有一个 OE 信号, 当该信号为高电平时 3PA1030 输出呈高阻态, 低电平则可以正常输出。

3PA1030 的时序图如下图所示:

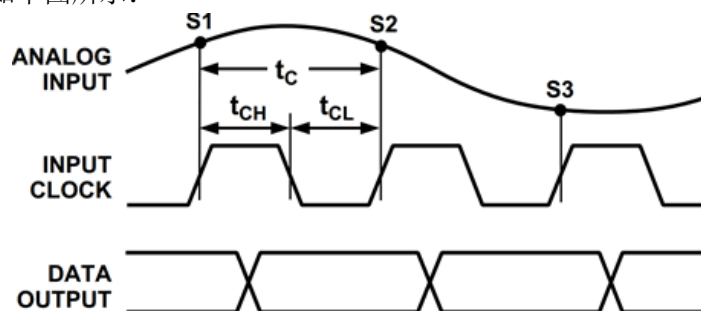


图 1.1.3 3PA1030 时序图

上图中, S1, S2, S3 分别为三个采样点, 可以看到, 芯片在时钟的上升沿采样。需要注意的是, 3PA1030 芯片的最大转换速度是 50MSPS, 即输入的时钟最大频率为 50MHz。

3PA1030 支持输入的模拟电压范围是 0V~2V, 0V 对应输出的数字信号为 0, 2V 对应输出的数字信号为 1023。而 ATK_DUAL_HS_AD 模块的输入电压范围是 -5V~+5V, 因此在 3PA1030 的模拟输入端增加电压衰减电路, 使 -5V~+5V 之间的电压转换成 0V~2V 之间。那么实际上对我们用户使用来说, 当 3PA1030 的模拟输入接口连接 -5V 电压时, AD 输出的数据为 0; 当 3PA1030 的模拟输入接口连接 +5V 电压时, AD 输出的数据为 1023。

当 3PA1030 模拟输入端接 -5V~+5V 之间变化的正弦波电压信号时, 其转换后的数据也是成正弦波波形变化, 转换波形如下图所示:

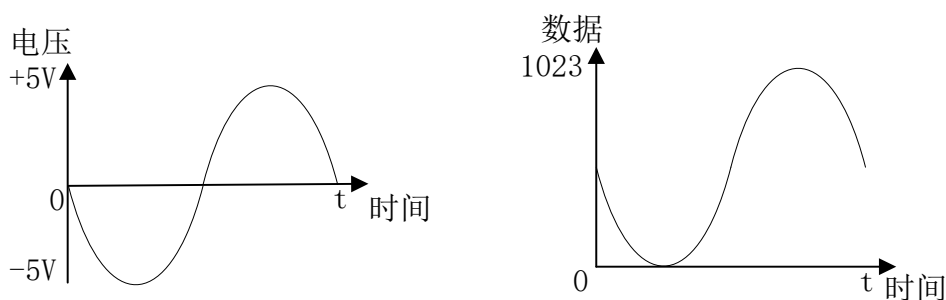


图 1.1.4 3PA1030 正弦波模拟电压值 (左)、数据 (右)

由上图可知, 输入的模拟电压范围在 -5V~5V 之间按照正弦波波形变化时, 最终得到的数据也是按照正弦波波形变化的。

1.2 硬件设计

ATK_DUAL_HS_AD 模块的实物图如下图所示:

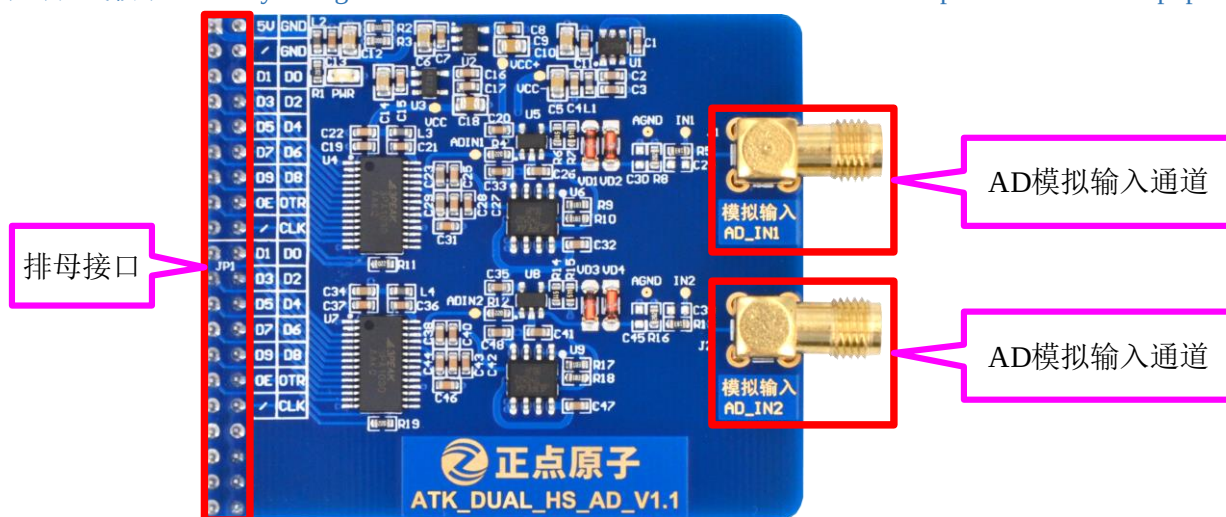


图 1.2.1 ATK_DUAL_HS_AD 模块实物图

ATK_DUAL_HS_AD 模块由 2 个型号为 3PA1030 的 AD 转换芯片组成。3PA1030 的原理图如下所示:

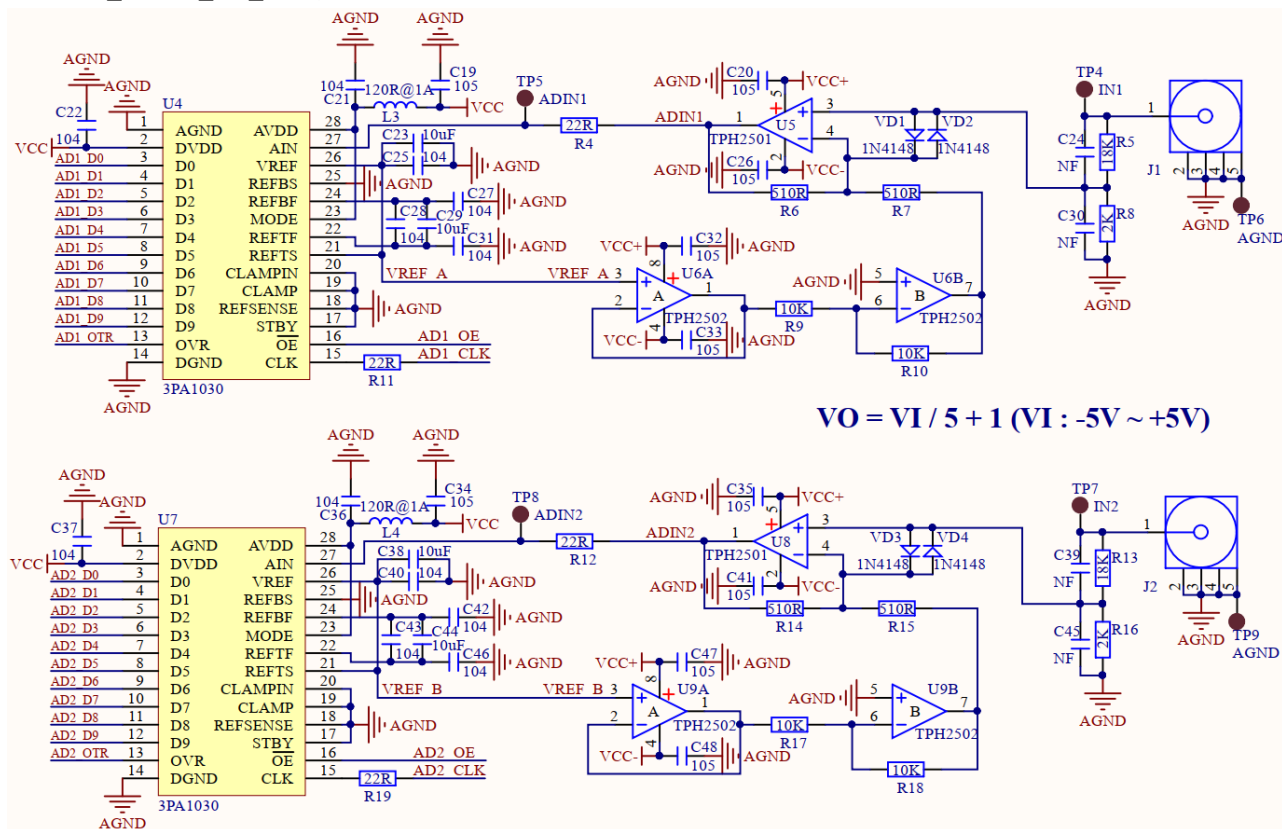


图 1.2.2 3PA1030 原理图

上下两个电路是一样的，我们以上面的电路为例。上图中输入的模拟信号 IN1（我们记作 VI）经过衰减电路后得到 ADIN1（我们记作 VO）信号，两个模拟电压信号之间的关系是 $VO = VI/5 + 1$ ，即当 $VI = 5V$ 时， $VO = 2V$ ； $VI = -5V$ 时， $VO = 0V$ 。

1.3 实验任务

本节实验任务是使用开发板及双路高速 AD 模块（ATK_DUAL_HS_AD 模块）实现双路模数的转换，并在 ILA 中查看波形。实验中我们将使用信号发生器来产生两个待采集的模拟信号，一个是频率为 1Mhz，幅值为 9V（±4.5V）的正弦波信号；另一个是频率为 1Mhz，幅值为 5V（±2.5V）的三角波信号。

1.4 程序设计

1.4.1 总体模块设计

根据实验任务可知, 本实验使用双路高速 AD 模块采集两路模拟波形信号, 经过模数转换 (ADC) 后, 将两路数字信号输出给 FPGA, 所以 FPGA 中需要一个 PLL IP 核, 来生成两路 AD 芯片的驱动时钟。

FPGA 接收到两路 AD 芯片输出的数字信号后, 由于不需要对数据进行处理, 所以使用 ILA 工具直接观察波形即可, 即 FPGA 中还需要一个 ILA IP 核。

至此, 整个设计架构基本完善, 综上所述, 我们可以绘制出如下所示的系统架构图:

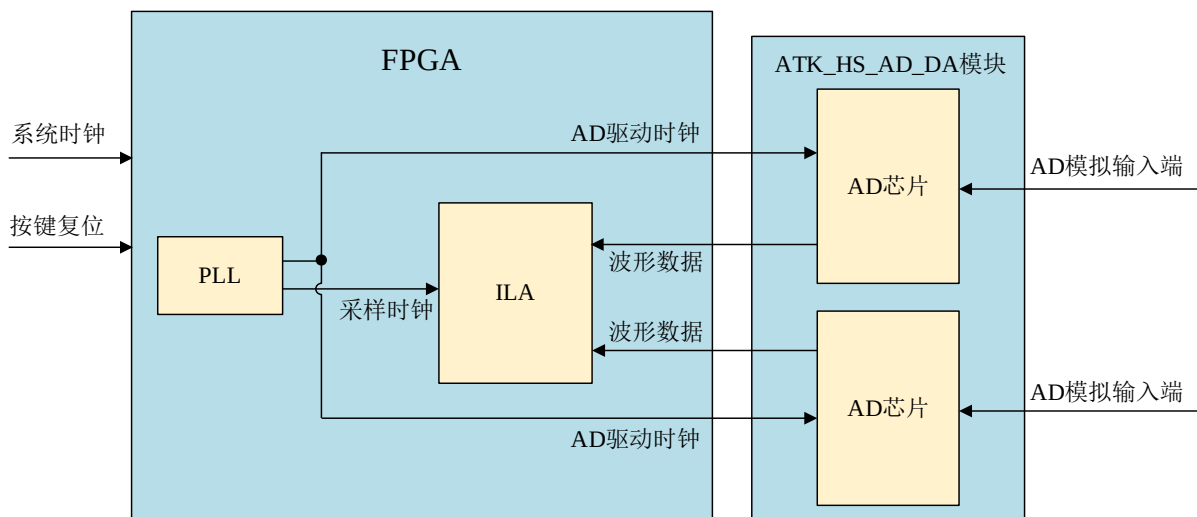


图 1.4.1 双路 AD 实验的系统架构图

接下来, 我们分别对上图中的各个模块的功能做进一步的说明。

PLL IP 核 (时钟模块): PLL IP 核输入时钟为 50MHz 系统时钟, 输出两路时钟, 分别为 50MHz 的 AD 芯片驱动时钟 (clk_50m) 和相位偏移 180 度的 ILA 采样时钟 (clk_50m_deg180)。

波形查看模块: 我们是使用 ILA 查看波形的, ILA 有五路输入信号, 分别为采样时钟信号 (clk_50m_deg180)、两路 10bit 位宽的模数转换后的数据信号 (ad_data_1 和 ad_data_2) 和两路 AD 芯片的超量程信号 (ad_otr_1 和 ad_otr_2)。

对于采样时钟, 理论上要与 AD 芯片驱动时钟的频率相同, 相位相反 (即相位偏移 180 度), 但是如大家在使用时发现在排除硬件问题后, 采集的数据仍然存在些许毛刺的话, 可以通过调整 ILA 采样时钟的相位来消除毛刺。

需要注意的是, 因为 3PA1030 芯片的 OE (高电平时 3PA1030 输出呈高阻态, 低电平则可以正常输出) 引脚被引到 IO 口上, 所以我们想要得到 AD 芯片输出的数据的话, 就需要将 OE 置低。

综上所述, 我们可以得到如下所示的系统框图:

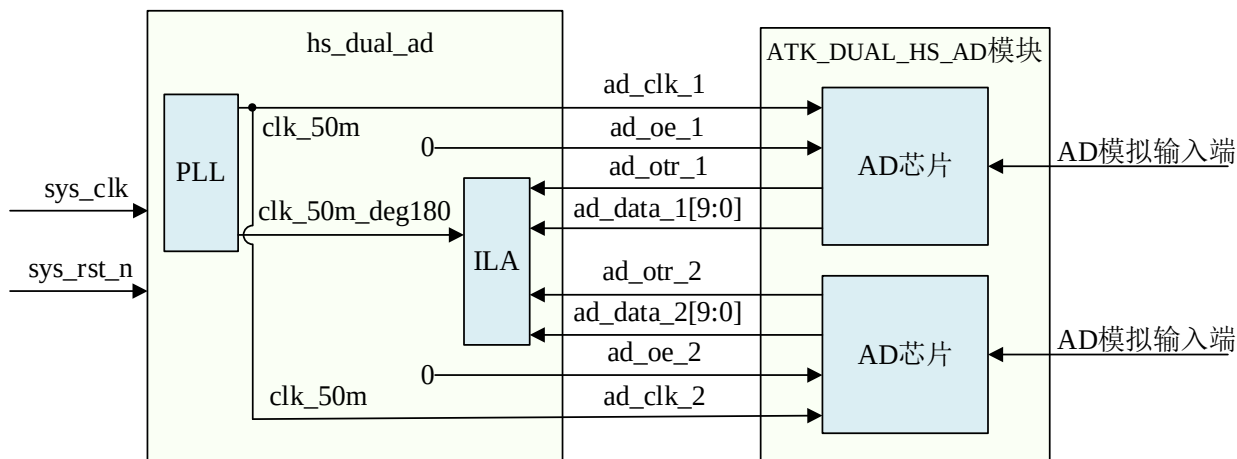


图 1.4.2 双路 AD 系统框图

顶层模块端口与功能描述如下表所示:

表 1.4.1 顶层模块端口与功能描述

信号名	位宽	方向	端口说明
sys_clk	1	输入	系统时钟, 50MHz
sys_rst_n	1	输入	系统复位按键, 低电平有效
ad_data_1	10	输入	接收的第一路10位ADC数据
ad_otr_1	1	输入	第一路ADC超出量程标志
ad_clk_1	1	输出	第一路ADC驱动时钟
ad_oe_1	1	输出	第一路ADC输出使能, 为低时正常输出
ad_data_2	10	输入	接收的第二路10位ADC数据
ad_otr_2	1	输入	第二路ADC超出量程标志
ad_clk_2	1	输出	第二路ADC驱动时钟
ad_oe_2	1	输出	第二路ADC输出使能, 为低时正常输出

1.4.2 PLL IP 核配置

本次实验我们需要使用 PLL IP 核来产生 50MHz 的 ADC 驱动时钟和 50MHz 相位偏移 180 度的 ILA 采样时钟。有关 PLL IP 核的内容, 还不熟悉的同学可以回顾下“IP 核之 PLL 实验”中的讲解, 这里我们仅对需要配置的参数进行展示说明。

“Clocking Options”选项卡的配置如下图所示:

Component Name **clk_wiz_0**

Clocking Options | Output Clocks | Port Renaming | MMCM Settings | Summary

Clock Monitor

☐ Enable Clock Monitoring

Primitive

☒ **MMCM** ☐ PLL

Clocking Features **Jitter Optimization**

☒ Frequency Synthesis ☐ Minimize Power ☒ Balanced

☒ Phase Alignment ☐ Spread Spectrum ☐ Minimize Output Jitter

☐ Dynamic Reconfig ☐ Dynamic Phase Shift ☐ Maximize Input Jitter filtering

☐ Safe Clock Startup

Dynamic Reconfig Interface Options

☒ AXI4Lite ☐ DRP ☐ Phase Duty Cycle Config ☐ Write DRP registers

Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
	Primary	clk_in1	50.000	10.000 - 800.000	UI	0.010	Single ended clock capable pir
☐	Secondary	clk_in2	100.000	30.000 - 60.000		0.010	Single ended clock capable pi

图 1.4.3 “Clocking Options” 选项卡配置

这里我们可以保持默认的 IP 核命名, “Primitive” 选择使用 MMCM, 输入时钟频率改为板载晶振的频率 50MHz。

“Output Clocks” 选项卡的配置如下图所示:

Clocking Options		Output Clocks		Port Renaming	MMCM Settings		Summary				
		Requested	Actual		Requested	Actual	Requested	Actual			
☒	clk_out1	clk_out1	50.000	⊗	50.000000	0.000	⊗	0.000	50.000	50.0	BUFG
☒	clk_out2	clk_out2	50.000	⊗	50.000000	180.000	⊗	180.000	50.000	50.0	BUFG
☐	clk_out3	clk_out3	100.000		N/A	0.000		N/A	50.000	N/A	BUFG
☐	clk_out4	clk_out4	100.000		N/A	0.000		N/A	50.000	N/A	BUFG
☐	clk_out5	clk_out5	100.000		N/A	0.000		N/A	50.000	N/A	BUFG
☐	clk_out6	clk_out6	100.000		N/A	0.000		N/A	50.000	N/A	BUFG
☐	clk_out7	clk_out7	100.000		N/A	0.000		N/A	50.000	N/A	BUFG

☐ USE CLOCK SEQUENCING

Clocking Feedback

Output Clock	Sequence Number
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Source

☒ Automatic Control On-Chip
☐ Automatic Control Off-Chip
☐ User-Controlled On-Chip
☐ User-Controlled Off-Chip

Signaling

☒ Single-ended
☐ Differential

Enable Optional Inputs / Outputs for MMCM/PLL

☐ reset
☐ power_down
☐ input_clk_stopped
☐ locked
☐ clkfbstopped

Reset Type

☒ Active High
☐ Active Low

图 1.4.4 “Output Clocks” 选项卡配置

这里我们勾选一路时钟输出，输出频率为 50MHz。另外，我们取消 PLL 复位信号（reset）和时钟锁定信号（locked）的勾选，其余选项保持不变，然后直接点击“OK”按钮完成 PLL IP 核的配置。

1.4.3 顶层模块设计

实验工程的各功能模块均已讲解完毕，接下来我们就需要根据总体模块设计中的讲解来将各个子模块例化到顶层模块中。

编写代码

顶层模块（hs_dual_ad）的代码如下所示：

```

1 module hs_dual_ad(
2     input    sys_clk    , //系统时钟
3     input    sys_rst_n  , //系统复位
4     //第一路 ADC
5     input    [9:0] ad_data_1 , //第一路 ADC 数据
6     input    ad_otr_1 , //第一路 ADC 输入电压超过量程标志
7     output   ad_clk_1 , //第一路 ADC 驱动时钟
8     output   ad_oe_1 , //第一路 ADC 输出使能
9     //第二路 ADC

```



```

10    input    [9:0]  ad_data_2 , //第二路 ADC 数据
11    input          ad_otr_2 , //第二路 ADC 输入电压超过量程标志
12    output         ad_clk_2 , //第二路 ADC 驱动时钟
13    output         ad_oe_2   //第二路 ADC 输出使能
14    );
15
16 //wire define
17 wire          clk_50m          ; //50MHz 时钟
18 wire          clk_50m_deg180   ; //相位偏移 180 的 50MHz 时钟
19
20 //*****
21 /**                               main code
22 //*****
23
24 assign ad_oe_1 = 1'b0;
25 assign ad_oe_2 = 1'b0;
26 assign ad_clk_1 = clk_50m;
27 assign ad_clk_2 = clk_50m;
28
29 //pll
30 clk_wiz_0 u_clk_wiz_0(
31     .clk_out1    (clk_50m          ),
32     .clk_out2    (clk_50m_deg180),
33     .clk_in1     (sys_clk          )
34 );
35
36 endmodule

```

1.5 下载验证

添加 ila IP 核进行在线调试

因为两路 ADC 的驱动时钟相同,所以我们只需要添加一个 ILA IP 核即可,这里我们将 ad_otr_1(1 位)、ad_data_1(10 位)、ad_otr_2(1 位)和 ad_data_2(10 位)这四个信号添加至观察列表中,添加 ILA IP 核的方法这里就不再赘述了,例化代码如下所示:

```

ila_0 u_ila_0 (
    .clk      (clk_50m_deg180 ), // input wire clk
    .probe0   (ad_otr_1       ), // input wire [0:0] probe0
    .probe1   (ad_data_1      ), // input wire [9:0] probe1
    .probe2   (ad_otr_2       ), // input wire [0:0] probe2
    .probe3   (ad_data_2      ) // input wire [9:0] probe3
);

```

引脚约束

接下来我们对引脚进行分配,并上板验证。本次实验需要用到 J2 扩展口来连接双路 AD 模块,本实验

中的管脚分配如下表所示:

表 1.5.1 双路AD实验管脚分配

信号名	方向	管脚	端口说明	电平标准
sys_clk	input	R4	系统时钟, 50MHz	LVC MOS33
sys_rst_n	input	U2	系统复位, 低有效	LVC MOS33
ad_data_1[0]	input	E19	第一路ADC输入数据	LVC MOS33
ad_data_1[1]	input	D19	第一路ADC输入数据	LVC MOS33
ad_data_1[2]	input	G22	第一路ADC输入数据	LVC MOS33
ad_data_1[3]	input	G21	第一路ADC输入数据	LVC MOS33
ad_data_1[4]	input	J22	第一路ADC输入数据	LVC MOS33
ad_data_1[5]	input	H22	第一路ADC输入数据	LVC MOS33
ad_data_1[6]	input	K22	第一路ADC输入数据	LVC MOS33
ad_data_1[7]	input	K21	第一路ADC输入数据	LVC MOS33
ad_data_1[8]	input	M21	第一路ADC输入数据	LVC MOS33
ad_data_1[9]	input	L21	第一路ADC输入数据	LVC MOS33
ad_otr_1	input	N22	第一路ADC模拟电压超出量程标志	LVC MOS33
ad_clk_1	output	E17	第一路ADC驱动时钟	LVC MOS33
ad_oe_1	output	M22	第一路ADC输出使能	LVC MOS33
ad_data_2[0]	input	F21	第二路ADC输入数据	LVC MOS33
ad_data_2[1]	input	F18	第二路ADC输入数据	LVC MOS33
ad_data_2[2]	input	L18	第二路ADC输入数据	LVC MOS33
ad_data_2[3]	input	M18	第二路ADC输入数据	LVC MOS33
ad_data_2[4]	input	G20	第二路ADC输入数据	LVC MOS33
ad_data_2[5]	input	H20	第二路ADC输入数据	LVC MOS33
ad_data_2[6]	input	J21	第二路ADC输入数据	LVC MOS33
ad_data_2[7]	input	J20	第二路ADC输入数据	LVC MOS33
ad_data_2[8]	input	M20	第二路ADC输入数据	LVC MOS33
ad_data_2[9]	input	N20	第二路ADC输入数据	LVC MOS33
ad_otr_2	input	N19	第二路ADC模拟电压超出量程标志	LVC MOS33

ad_clk_2	output	T20	第二路ADC驱动时钟	LVC MOS33
ad_oe_2	output	N18	第二路ADC输出使能	LVC MOS33

对应的 XDC 约束语句如下所示:

#时钟约束

```
create_clock -period 20.000 -name sys_clk [get_ports sys_clk]
```

#IO 引脚约束

```
set_property -dict {PACKAGE_PIN R4 IOSTANDARD LVC MOS33} [get_ports sys_clk]
```

```
set_property -dict {PACKAGE_PIN U2 IOSTANDARD LVC MOS33} [get_ports sys_rst_n]
```

```
set_property -dict {PACKAGE_PIN E17 IOSTANDARD LVC MOS33} [get_ports ad_clk_1]
```

```
set_property -dict {PACKAGE_PIN M22 IOSTANDARD LVC MOS33} [get_ports ad_oe_1]
```

```
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVC MOS33} [get_ports ad_otr_1]
```

```
set_property -dict {PACKAGE_PIN E19 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[0]}]
```

```
set_property -dict {PACKAGE_PIN D19 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[1]}]
```

```
set_property -dict {PACKAGE_PIN G22 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[2]}]
```

```
set_property -dict {PACKAGE_PIN G21 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[3]}]
```

```
set_property -dict {PACKAGE_PIN J22 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[4]}]
```

```
set_property -dict {PACKAGE_PIN H22 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[5]}]
```

```
set_property -dict {PACKAGE_PIN K22 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[6]}]
```

```
set_property -dict {PACKAGE_PIN K21 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[7]}]
```

```
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[8]}]
```

```
set_property -dict {PACKAGE_PIN L21 IOSTANDARD LVC MOS33} [get_ports {ad_data_1[9]}]
```

```
set_property -dict {PACKAGE_PIN T20 IOSTANDARD LVC MOS33} [get_ports ad_clk_2]
```

```
set_property -dict {PACKAGE_PIN N18 IOSTANDARD LVC MOS33} [get_ports ad_oe_2]
```

```
set_property -dict {PACKAGE_PIN N19 IOSTANDARD LVC MOS33} [get_ports ad_otr_2]
```

```
set_property -dict {PACKAGE_PIN F21 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[0]}]
```

```
set_property -dict {PACKAGE_PIN F18 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[1]}]
```

```
set_property -dict {PACKAGE_PIN L18 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[2]}]
```

```
set_property -dict {PACKAGE_PIN M18 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[3]}]
```

```
set_property -dict {PACKAGE_PIN G20 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[4]}]
```

```
set_property -dict {PACKAGE_PIN H20 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[5]}]
```

```
set_property -dict {PACKAGE_PIN J21 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[6]}]
```

```
set_property -dict {PACKAGE_PIN J20 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[7]}]
```

```
set_property -dict {PACKAGE_PIN M20 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[8]}]
```

```
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVC MOS33} [get_ports {ad_data_2[9]}]
```

上板验证

将双路高速 AD 模块插入开发板的 J2 扩展口, 连接时注意扩展口电源引脚方向和双路高速 AD 模块的电源引脚方向要一致, 如下图所示:

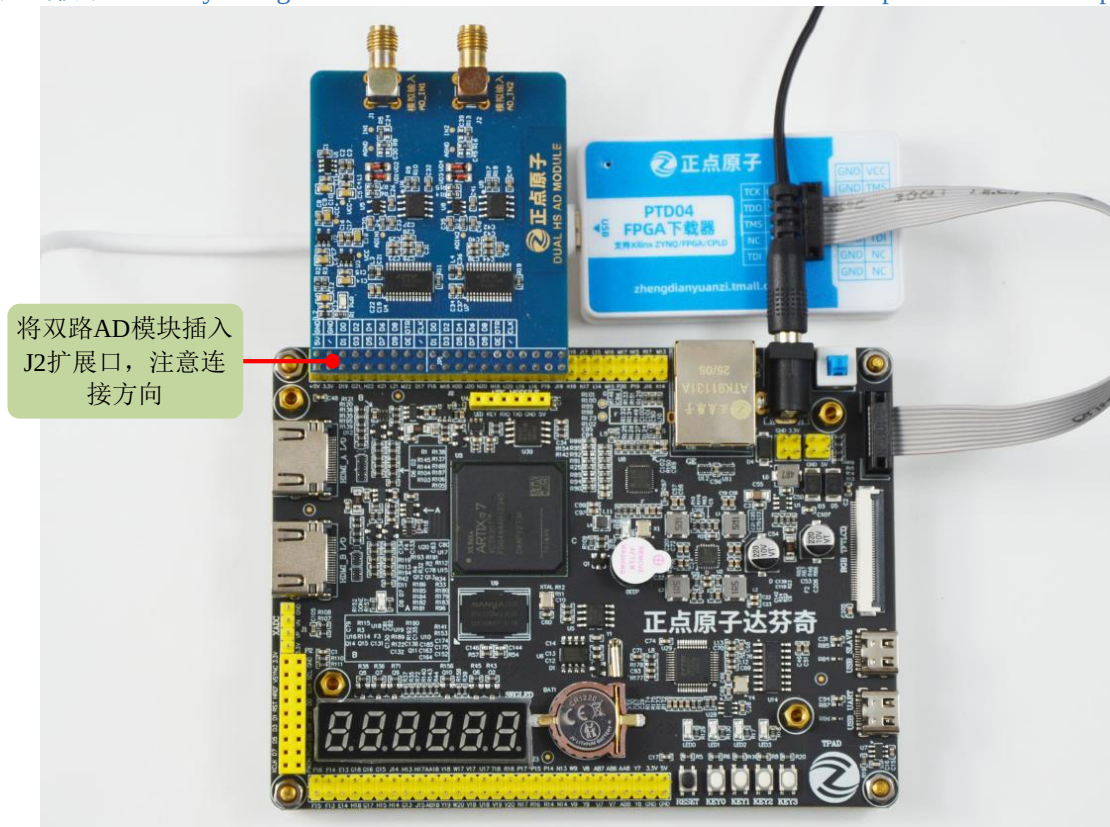


图 1.5.1 双路高速 AD 模块实物连接图

然后使用 SMA 转 BNC 线将信号发生器与 AD 模块进行连接, 如下图所示:

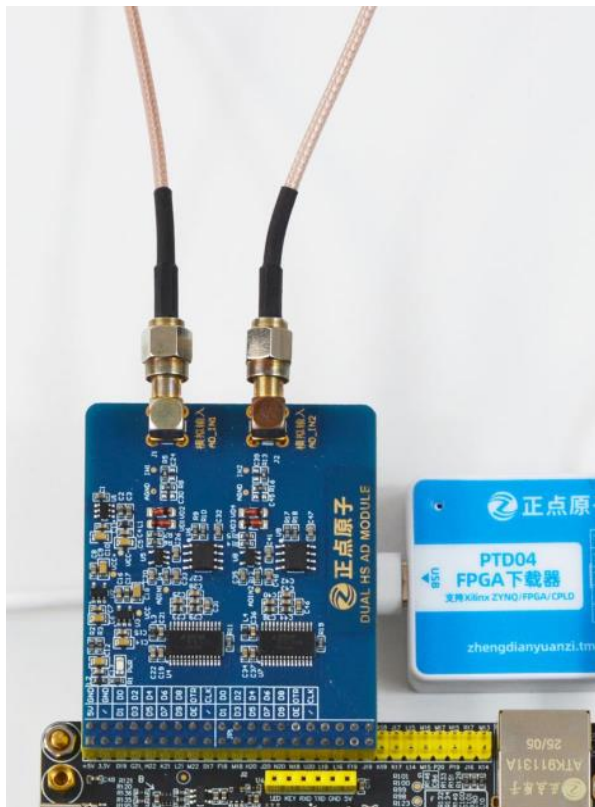


图 1.5.2 信号发生器与双路 AD 模块的连接图

信号发生器设置如下所示:



图 1.5.3 信号发生器设置

接着将下载器一端连接电脑，另一端与开发板上的 JTAG 端口连接，最后连接电源线并打开电源开关，将工程生成的比特流文件下载到开发板中。

运行程序后在 ILA 中观察 ad_data 数据的变化，观察到的波形如下图所示：

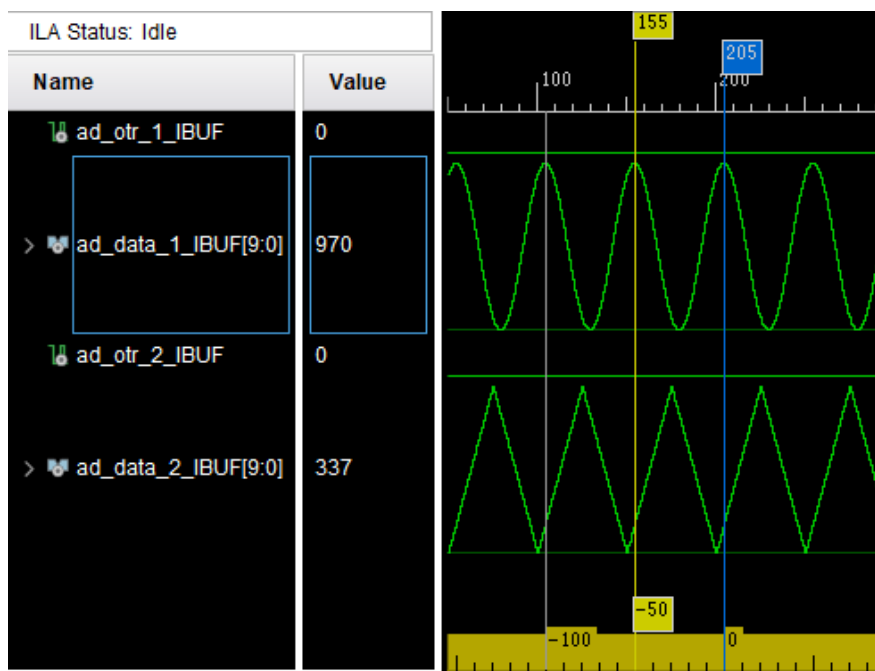


图 1.5.4 AD 数据在 ILA 中显示的波形图 1

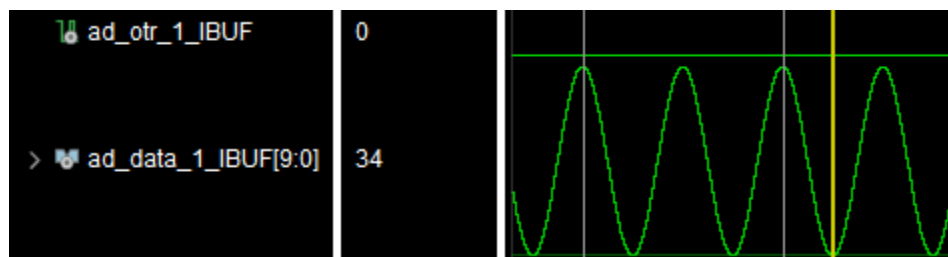


图 1.5.5 AD 数据在 ILA 中显示的波形图 2

由图 1.5.4 和图 1.5.5 可知，输入的第一路 ad_data 数据为正弦波，且一个正弦波波形周期为 50 个采样

时钟 (50MHz) 周期, 即正弦波波形频率为 1MHz。另外从图中我们还可以知道正弦波的峰值数据为 970, 谷值数据为 34, 再结合图 1.1.4 可大致估计峰值数据对应 4.5V 左右, 谷值数据对应 -4.5V 左右, 即正弦波信号的幅值为 9V。

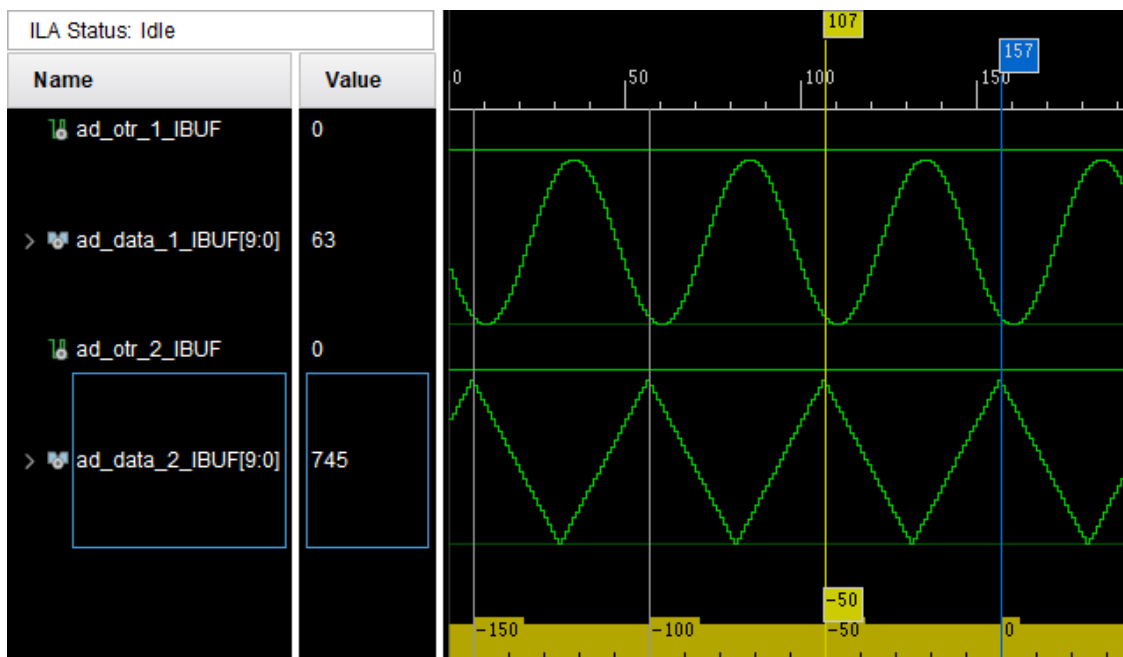


图 1.5.6 AD 数据在 ILA 中显示的波形图 3

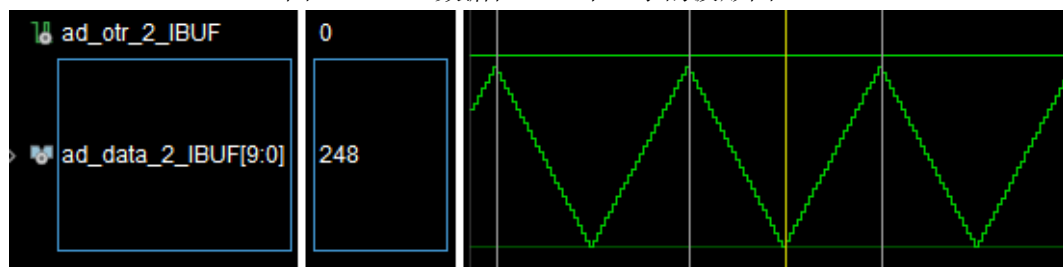


图 1.5.7 AD 数据在 ILA 中显示的波形图 4

由图 1.5.6 和图 1.5.7 可知, 输入的第二路 ad_data 数据为三角波, 且一个三角波波形周期为 50 个采样时钟 (50MHz) 周期, 即三角波波形频率为 1MHz。另外从图中我们还可以知道三角波的峰值数据为 745, 谷值数据为 248, 再结合图 1.1.4 可大致估计峰值数据对应 2.5V 左右, 谷值数据对应 -2.5V 左右, 即三角波信号的幅值为 5V。

综上可知采集到的正弦波和三角波的频率和幅值与我们通过信号发生器产生的信号相同, 说明双路 AD 实验验证成功。

1.6 本章总结

经过本章的学习, 我们学会了如何实现开发板与双路高速 AD 模块之间的数据交互。需要注意的是, 通常情况下我们只需要给出 ADC 驱动时钟即可获取到 ADC 数据, 例如“高速 AD/DA 实验”中使用的 AD 芯片。而本次实验中使用的 AD 芯片具有使能数据输出的功能, 当我们不想接收 AD 芯片输出的数据时就可以将芯片的 OE 引脚置高 (1), 这样芯片的输出就都为高阻态了; 当我们想要接收 AD 芯片输出的数据时就可以将芯片的 OE 引脚置低 (0), 这样芯片的输出就正常了, 通常情况下我们将 OE 恒置为低即可。

1.7 拓展训练

本章使用的双路高速 AD 模块支持的最大转换速率为 50MSPS (即驱动时钟最大为 50MHz), 大家可以在不超出最大转换速率的情况下任意改变 ADC 驱动时钟的频率, 以观察在不同驱动时钟频率下采集同一个波形信号时有什么区别。